Rethinking Model Superiority in Long Term Time Series Forecasting

March 11, 2026

1 Computational Efficiency Analysis

Beyond forecasting accuracy, practical time series forecasting systems must also consider computational efficiency, including model size, training cost, and inference latency. To provide a comprehensive comparison, we analyze the computational characteristics of the evaluated models from three perspectives: (i) model size and parameter scale, (ii) training efficiency, and (iii) inference efficiency. The detailed statistics are summarized in Figures 1, 2, and 3.

Figure 1 compares the parameter scale and storage footprint of the considered architectures. Linear based models exhibit extremely compact designs. For example, SparseTSF contains only 17,975 parameters (0.08 MB), while DLinear requires 209,520 parameters (0.8 MB). These models rely on simple linear mappings along the temporal dimension, allowing them to achieve strong efficiency with minimal parameter overhead. In contrast, more expressive architectures significantly increase the parameter scale. Transformer-based models such as iTransformer and PatchTST contain 6.7M and 1.5M parameters, respectively, while graph or interaction based models such as DiM reach over 15M parameters. The most extreme case is FSMLP, which contains more than 98M parameters and occupies approximately 375 MB of storage. This result illustrates how different temporal and channel modeling strategies can lead to substantial variations in model complexity.

The computational implications of these architectural choices are further reflected in the training efficiency shown in Figure 2. Lightweight linear models again exhibit the lowest computational cost, with DLinear requiring only 79.8 seconds of training time and relatively low peak memory usage. Models introducing moderate nonlinear transformations, such as FTMLP or SparseTSF, slightly increase the training cost but remain efficient compared with more complex architectures. On the other hand, models that rely on deeper representations or interaction modeling require significantly larger computational resources. For instance, TimeMixer shows the highest training memory consumption (16.6 GB) and the longest training time (over 700 seconds), indicating that multi component temporal mixing mechanisms may introduce substantial computational overhead. Similarly, transformer-based models such as PatchTST

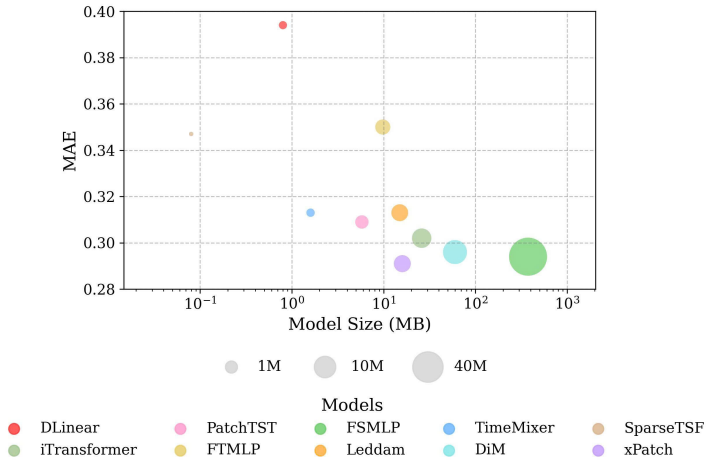

Figure 1: Comparison of model size and parameter scale across different forecasting architectures.

and iTransformer require considerably higher training memory due to attention operations and deeper architectural stacks.

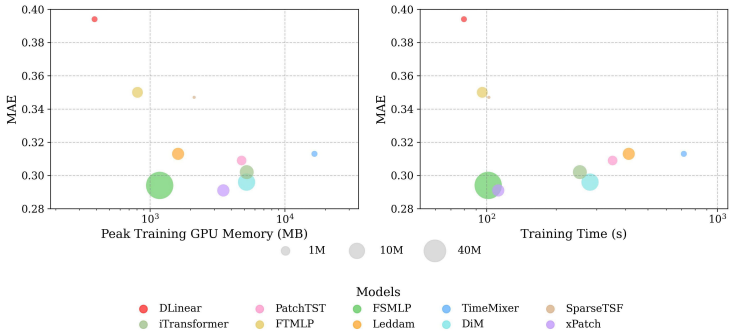

Figure 2: Training efficiency comparison of forecasting models in terms of training time and GPU memory usage.

Figure 3 further evaluates the runtime efficiency during inference. The results reveal a similar trend: simpler architectures generally provide faster inference and lower memory requirements. DLinear and SparseTSF achieve inference times of approximately 22-25 seconds for 2789 samples, demonstrating strong suitability for real time forecasting scenarios. In contrast, more complex architectures incur noticeably higher inference latency. For example, DiM requires nearly 238 seconds for inference, while transformer based models also show increased latency compared with lightweight alternatives. Additionally, some models such as PatchTST exhibit relatively high inference memory us-

age (over 1.2 GB), which may limit their deployment in resource constrained environments.

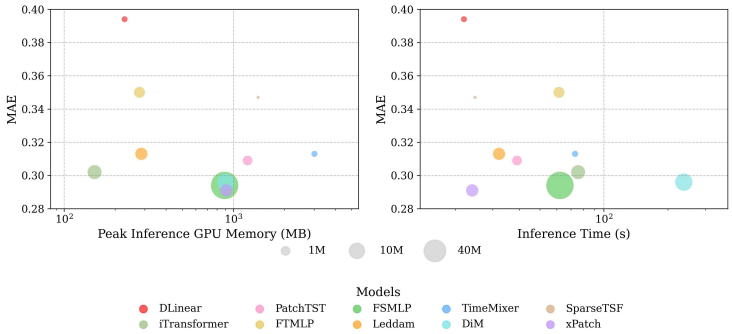

Figure 3: Inference efficiency comparison of forecasting models in terms of runtime and memory consumption.

Overall, the results highlight a clear trade off between representational capacity and computational efficiency. Linear and lightweight architectures offer excellent scalability and deployment efficiency, while more complex models provide stronger modeling flexibility at the cost of significantly higher computational requirements. These observations suggest that model selection in practical forecasting systems should carefully balance predictive performance with computational constraints, particularly in large-scale or real-time applications.